

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: MLA03 COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO4: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Mock Interview

Duration: 1hr

-
1. A machine learning engineer applies Reinforcement Learning to solve sequential decision-making problems. Explain how an agent learns optimal behavior through interaction with the environment. **(5 Marks)**
 2. If you are asked to design a Reinforcement Learning solution for a real-time system such as ride-sharing or delivery optimization, explain how you would formulate the problem using a Markov Decision Process. **(5 Marks)**
 3. Compare model-free and model-based Reinforcement Learning approaches. Explain their working principles and how they handle uncertainty. **(5 Marks)**
 4. A recommendation system using Reinforcement Learning is underperforming. Explain how reward sparsity and hyperparameter tuning can be diagnosed and

improved.

(5 Marks)

5. If you are implementing an RL-based game-playing AI, describe how you would design the learning pipeline and ensure training stability. (5 Marks)
6. A company wants to use Reinforcement Learning for dynamic pricing. Explain how the agent learns from interaction and improves pricing decisions over time. (5 Marks)
7. If an RL model is not converging properly, explain how you would identify and fix issues related to learning rate and exploration strategy. (5 Marks)
8. A robotics system uses Reinforcement Learning for navigation. Explain how the environment and agent interaction leads to optimal path learning. (5 Marks)
9. If you are asked to improve an RL model with delayed rewards, explain techniques to handle credit assignment problems. (5 Marks)
10. A financial firm uses Reinforcement Learning for trading decisions. Explain how uncertainty and risk can be handled in the learning process. (5 Marks)

Code of Conduct:

*I **Dasari Haritha(192425254)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Dasari Haritha

MOCK INTERVIEW

1. How does an agent learn optimal behavior through interaction with the environment?

Reinforcement Learning is a machine learning approach where an **agent learns by interacting with an environment** and receiving feedback in the form of rewards.

The basic process is:

State → Action → Environment → Reward + Next State → Policy Update

- The **agent** observes the current state of the environment.
- Based on its current **policy**, it selects an action.
- The environment executes the action and produces a **reward** and a new state.
- The agent uses this experience to determine whether the action was good or bad.
- Over repeated interactions, the agent improves its policy to maximize the **cumulative future reward**.

For example, in a game, the state may represent the current game board, the actions may be left/right/jump, and the reward may be positive for winning or negative for losing.

The agent must also balance:

- **Exploration** – trying new actions to discover better strategies.
- **Exploitation** – choosing actions that are already known to provide good rewards.

The ultimate objective is to learn an optimal policy:

$\pi^*(a|s)$ = action that maximizes expected cumulative reward.

Interview conclusion:

“So, unlike supervised learning, where the correct answer is provided, an RL agent learns through trial and error using rewards from the environment.”

2. Formulate a ride-sharing or delivery optimization problem using an MDP

A real-time ride-sharing or delivery system can be formulated as a **Markov Decision Process (MDP)** using five main components:

$$\text{MDP} = (\text{S}, \text{A}, \text{P}, \text{R}, \gamma)$$

1. State (S)

The state represents the current situation of the system.

For ride-sharing, it could include:

- Driver locations
- Passenger locations
- Current traffic conditions
- Available vehicles
- Number of pending requests
- Driver status

2. Action (A)

The action represents what the RL agent can decide.

Examples:

- Assign driver to passenger

- Choose delivery vehicle
- Select delivery route
- Reposition an available driver

3. Transition Probability (P)

It represents the probability of moving from the current state to a new state after taking an action.

For example:

Current traffic + selected route → predicted next traffic/travel state

Because traffic and customer requests are uncertain, transitions are generally stochastic.

4. Reward (R)

The reward defines the objective.

For example:

Positive reward:

- Fast delivery
- Successful ride assignment
- High customer satisfaction

Negative reward:

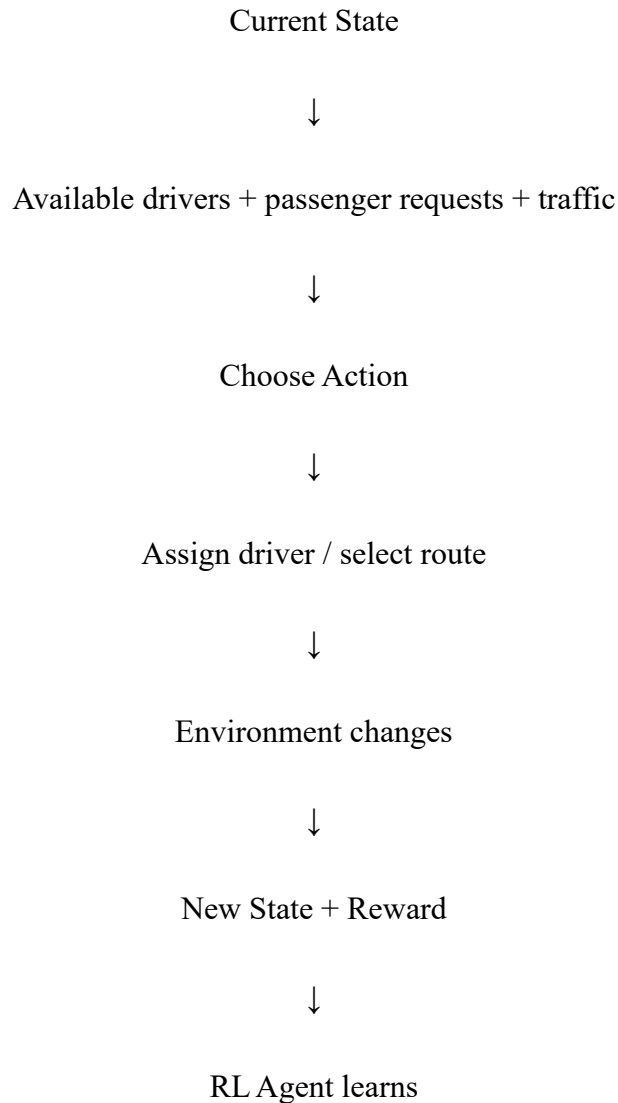
- Long waiting time
- High fuel consumption
- Late delivery
- Unassigned requests

5. Discount Factor (γ)

The discount factor determines how much importance is given to future rewards.

A high γ means the system considers **long-term consequences** more strongly.

Example



Interview conclusion:

“I would formulate ride-sharing as an MDP by defining the current traffic and vehicle situation as the state, driver assignments as actions, travel/request dynamics as transitions, and waiting time, cost and successful service as rewards.”

3. Compare Model-Free and Model-Based Reinforcement Learning

The major difference is whether the RL agent **learns an explicit model of the environment**.

Feature	Model-Based RL	Model-Free RL
Environment model	Learns an explicit model	Does not explicitly learn one
Learning	Model + planning	Directly learns policy/value
Planning	Uses planning/simulation	Usually does not perform explicit planning
Future prediction	Predicts future states/rewards	Learns mainly from observed experience
Sample efficiency	Generally higher	Generally lower
Computation	More computation during planning	Usually less planning computation
Uncertainty	Represents/predicts transition uncertainty	Learns uncertainty indirectly through experience
Examples	Dyna, MPC-based RL	Q-Learning, SARSA, DQN

Model-Based RL

The agent learns:

- **Transition model:**

$$P(s' | s, a)$$

- **Reward model:**

$$R(s, a)$$

It then uses the learned model to **simulate possible future actions** and choose a promising action.

Environment



Learn Model



Predict Future



Planning



Best Action

Model-Free RL

The agent does not explicitly construct the environment model. Instead, it learns a:

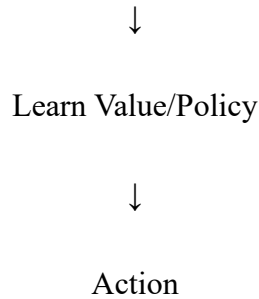
- **Policy**, or
- **Value/Q-function**

directly from experience.

Environment



Experience



Handling uncertainty

Model-Based RL: uncertainty can be represented through transition probabilities or multiple predicted outcomes. However, an inaccurate model can produce poor plans.

Model-Free RL: uncertainty is handled through repeated interaction, exploration, and learning from rewards. It does not explicitly predict the environment.

Interview conclusion:

“Model-based RL is useful when we want to plan using a learned model and make better use of limited experience, while model-free RL is simpler because it learns the policy or value function directly from interaction.”

4. Recommendation system underperforming: reward sparsity and hyperparameter tuning

If an RL-based recommendation system is underperforming, I would first determine whether the problem is related to the **reward design**, insufficient feedback, or incorrect hyperparameters.

A. Diagnose Reward Sparsity

Reward sparsity occurs when the agent receives meaningful rewards **very rarely**.

For example, if a recommendation receives a reward only when a user purchases a product, most interactions may produce zero reward.

Recommendation



User Interaction



No Purchase → Reward = 0

No Purchase → Reward = 0

No Purchase → Reward = 0

Purchase → Reward = +1

This makes learning difficult because the agent receives little useful feedback.

How to improve it

Use more informative intermediate rewards.

For example:

- Impression → small reward
- Click → positive reward
- Long engagement → larger reward
- Add to cart → higher reward
- Purchase → highest reward

This creates a **reward signal at multiple stages**.

However, reward design should avoid encouraging undesirable behavior, such as maximizing clicks while reducing actual user satisfaction.

B. Diagnose Hyperparameters

Important RL hyperparameters include:

- Learning rate
- Discount factor γ
- Exploration rate ϵ
- Batch size
- Replay-buffer size
- Target-network update frequency

I would monitor:

- Average reward
- Click-through rate
- Conversion rate
- Episode performance
- Training loss
- Exploration/exploitation behavior

Then perform controlled tuning using methods such as:

Grid Search / Random Search / Bayesian Optimization

Example

If the agent is not exploring enough:

Increase ϵ

If learning is unstable:

Reduce learning rate

If future rewards are important:

Increase γ

Interview conclusion:

“I would first inspect the reward distribution to identify sparsity, then redesign the reward carefully and tune the major RL hyperparameters while monitoring both training metrics and real recommendation metrics.”

5. Design an RL-based game-playing AI and ensure training stability

For an RL-based game-playing AI, I would design the system as a complete pipeline:

Game Environment



State Representation



RL Agent



Action Selection



Game Environment



Reward



Experience



Training



Step 1: Define the State

The state should represent the current game situation.

For example:

- Player position
- Enemy position
- Health
- Score
- Available actions
- Game objects

For image-based games, a neural network can process game frames.

Step 2: Define Actions

Actions could be:

Move Left

Move Right

Jump

Attack

Defend

Step 3: Design Rewards

Rewards should encourage desirable behavior.

Example:

- Winning $\rightarrow +100$
- Defeating enemy $\rightarrow +10$
- Collecting useful item $\rightarrow +5$
- Losing $\rightarrow -100$
- Unnecessary action $\rightarrow$ small penalty

Step 4: Choose the RL Algorithm

For a discrete-action game, I could use:

- **DQN**
- Double DQN
- Dueling DQN

For continuous actions, algorithms such as:

- PPO
- SAC
- Actor-Critic methods

may be suitable depending on the problem.

Ensuring Training Stability

I would use several techniques:

1. Experience Replay

Store experiences:

(State, Action, Reward, Next State)

and randomly sample batches during training. This reduces correlation between consecutive experiences.

2. Target Network

Maintain a separate target network and update it periodically to make learning more stable.

3. Reward Scaling/Clipping

Keep extremely large rewards from destabilizing training.

4. Controlled Exploration

Start with more exploration and gradually reduce it so the agent eventually exploits learned strategies.

5. Monitor Training

Track:

- Average reward
- Win rate
- Episode length
- Loss

- Exploration rate

6. Evaluation

Periodically evaluate the agent **without exploration** to determine whether its actual gameplay performance is improving.

Interview conclusion:

“I would build the pipeline around state representation, action selection, reward generation, experience collection and neural-network training. To maintain stability, I would use experience replay, target networks, controlled exploration, appropriate reward scaling and continuous evaluation using metrics such as average reward and win rate.”

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problemsolving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides wellstructured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately